DAY 31

Model Evaluation, Cross-Validation and Hyperparameter Tuning

60-Day Data Science Learning Journey

Learning outcomes
Interpret classification metrics, explain five-fold cross-validation, distinguish parameters from hyperparameters, use GridSearchCV and RandomizedSearchCV, and evaluate a selected model without tuning on the final test set.

This guide includes worked calculations, diagrams, a Python workflow, code explanations, common mistakes, exercises with solutions, revision notes, and a quiz.

1. Why model evaluation matters

A machine learning model learns patterns from examples. Evaluation estimates how well those patterns work on records not used to fit the model. A model may perform well on training data but poorly on new data. This is often called overfitting.

Training set: fits the model. **Validation or cross-validation data:** helps compare models and settings. **Final test set:** remains untouched until decisions are complete, then gives a final performance estimate.

Recommended workflow: reserve a test set first, use cross-validation on training data to select settings, refit the selected model, then evaluate on the untouched test set.

2. Confusion matrix and classification metrics

Assume positive means a student passes. These numbers are an illustrative example, not the output of the practice model.

	Predicted pass	**Predicted fail**
Actually pass	TP = 45	FN = 5
Actually fail	FP = 10	TN = 40

TP: predicted pass and actually passed. **TN:** predicted fail and actually failed. **FP:** predicted pass but actually failed. **FN:** predicted fail but actually passed.

Accuracy

Accuracy = (TP + TN) / total = (45 + 40) / 100 = **85%**. It measures the share of all predictions that are correct.

Precision

Precision = TP / (TP + FP) = 45 / 55 = **81.82%**. Of all predicted positives, how many were actually positive? Precision matters when false positives are costly.

Recall

Recall = TP / (TP + FN) = 45 / 50 = **90%**. Of all actual positives, how many did the model identify? Recall matters when missing positive cases is costly.

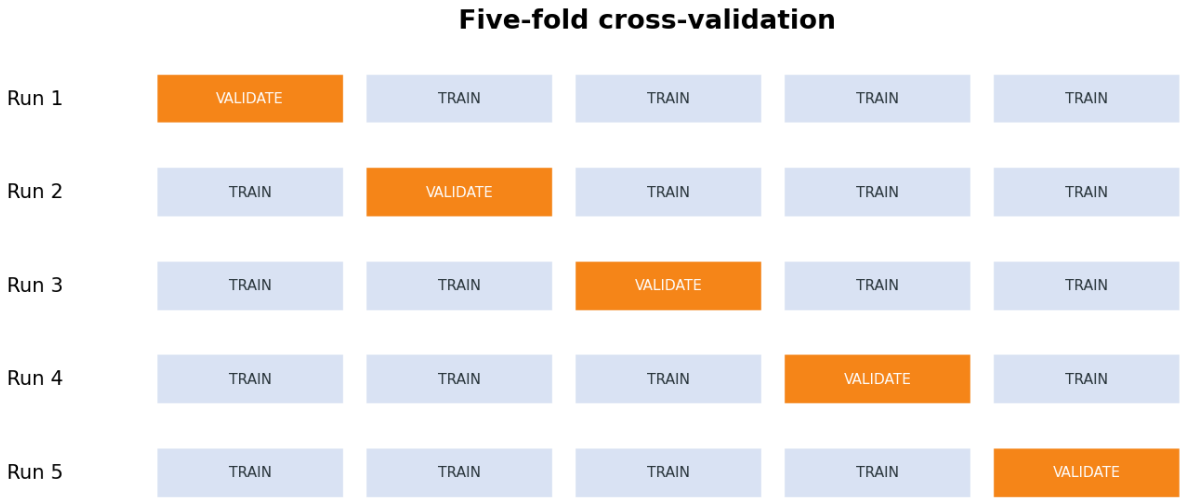
F1-score

F1 = 2 × (precision × recall) / (precision + recall). For precision 0.8182 and recall 0.90, F1 is approximately **85.71%**. F1 balances precision and recall using their harmonic mean.

No single metric is best for every problem. Accuracy can mislead when classes are imbalanced. Consider the class distribution and the real cost of each error.

3. Cross-validation

In 5-fold cross-validation, divide the training data into five parts. Train on four folds and validate on the remaining fold. Repeat five times so each fold validates once.



Example validation accuracies: 82%, 86%, 84%, 88%, and 85%. Mean = (82+86+84+88+85)/5 = **85%**. The spread also matters because a large spread can mean performance depends strongly on the particular split.

StratifiedKFold helps preserve class proportions in classification folds. For time-series prediction, random folds can leak future information, so use a time-aware validation strategy.

Cross-validation does not create independent new data. It reuses different partitions of the training set. Keep the final test set out of tuning.

4. Parameters vs hyperparameters

Parameters are learned during training, such as weights in a linear model. Hyperparameters are settings chosen to control learning.

n_estimators: number of trees in a Random Forest. More trees can improve stability but increase computation.
max_depth: maximum tree depth. Limiting depth can reduce complexity.
min_samples_split: minimum samples needed to split a node.
min_samples_leaf: minimum samples in a leaf.

5. GridSearchCV vs RandomizedSearchCV

GridSearchCV tests every combination in the supplied grid. Two values for n_estimators and three values for max_depth make 2 × 3 = 6 combinations. With five-fold CV, this requires 30 fits plus a final refit by default.

RandomizedSearchCV samples a chosen number of combinations from lists or distributions. It can explore a large search space more cheaply, but it might not sample the best possible combination.

6. Python practice

The companion file **Day-31-Python-Practice.py** contains a complete runnable example using scikit-learn's built-in Breast Cancer Wisconsin dataset for educational classification practice only. It is not a medical diagnostic tool.

Install dependencies: `pip install scikit-learn scipy`
Run: `python Day-31-Python-Practice.py`

Code walkthrough

train_test_split(..., stratify=y) creates a holdout set while helping preserve class proportions. **StratifiedKFold** defines five folds. **cross_val_score** returns a score for each fold. **GridSearchCV** tests every supplied combination. **best_params_** gives the best settings found, **best_score_** the mean cross-validation score, and **best_estimator_** the selected fitted model.

The script compares GridSearchCV and RandomizedSearchCV, then reports accuracy, precision, recall, F1-score, confusion matrix, and classification report on held-out test data. Exact scores can vary with library versions.

Important: Accuracy is used as the search metric to keep the example straightforward. In real projects, select metrics according to the costs of errors and the goal of the application.

7. Common mistakes

1. Evaluating only on training data can hide overfitting.
2. Tuning repeatedly on the final test set makes it part of model selection.
3. Accuracy alone may hide poor performance on a minority class.
4. Data leakage happens when information from validation or test data influences training. Use a Pipeline for learned preprocessing inside cross-validation.
5. A high cross-validation score does not guarantee future performance.
6. Randomly splitting time-series data may let future information influence the past.

8. Exercises with solutions

Exercise 1: Calculate metrics for TP=32, TN=50, FP=8, FN=10.

Solution: Total = 100. Accuracy = (32+50)/100 = 82%. Precision = 32/(32+8) = 80%. Recall = 32/(32+10) ≈ 76.19%. F1 ≈ 78.05%.

Exercise 2: A grid has four n_estimators values and three max_depth values. How many combinations and fits with five-fold CV?

Solution: 4 × 3 = 12 combinations. 12 × 5 = 60 CV fits, plus a final refit by default.

Exercise 3: When is RandomizedSearchCV useful?

Solution: When the search space is large and exhaustive grid search is too expensive. It samples a fixed number of candidates.

Exercise 4: Why keep the final test set separate?

Solution: To prevent test-set information from influencing model selection and preserve a more honest final estimate.

Exercise 5: A dataset is 99% negative. Why can 99% accuracy be misleading?

Solution: Predicting every record as negative could reach 99% accuracy while identifying no positive cases. Inspect precision, recall, F1, and the confusion matrix.

9. Revision notes

• Accuracy = correct predictions / all predictions.
• Precision = TP / (TP + FP).
• Recall = TP / (TP + FN).
• F1 is the harmonic mean of precision and recall.
• Cross-validation compares performance across folds.
• Stratified folds help preserve class proportions.
• GridSearchCV tests all supplied combinations.
• RandomizedSearchCV samples a set number of candidates.
• Keep the final test set out of model selection.
• Avoid leakage and choose metrics for the real task.

10. Quick quiz with answers

1. What is cross-validation for? Estimating performance across multiple data splits.
2. Which metric measures the fraction of predicted positives that are correct? Precision.
3. What does GridSearchCV do? Tests the supplied hyperparameter combinations using cross-validation.
4. Why keep the test set separate? To avoid using final evaluation data to select the model.
5. One advantage of RandomizedSearchCV? It can explore large search spaces with fewer sampled combinations.

Completion checklist

■ I can explain TP, TN, FP, and FN.
■ I can calculate accuracy, precision, recall, and F1.
■ I understand five-fold cross-validation.
■ I can explain GridSearchCV and RandomizedSearchCV.
■ I ran the Python script and inspected final test metrics.

Strong model evaluation is not only about achieving a high score. It is about understanding how reliable that score is and whether the model is useful for the real problem.